Introduction to Kubernetes

Class no:1

Kubernetes

Kubernetes (also known as **K8s**) is an open-source container orchestration platform. Think of it as the conductor of an orchestra, where your software containers (like Docker) are the musicians. Instead of managing each container manually, Kubernetes automates their deployment, scaling, management, and networking so everything runs smoothly without human intervention.

Here is a breakdown of how it works and the key components that make it tick.

The Architecture: Control Plane vs. Worker Nodes

A Kubernetes setup is called a **Cluster**. A cluster is divided into two main parts: the **Control Plane** (the brains) and the **Worker Nodes** (the muscle).

1. The Control Plane (The Brains)

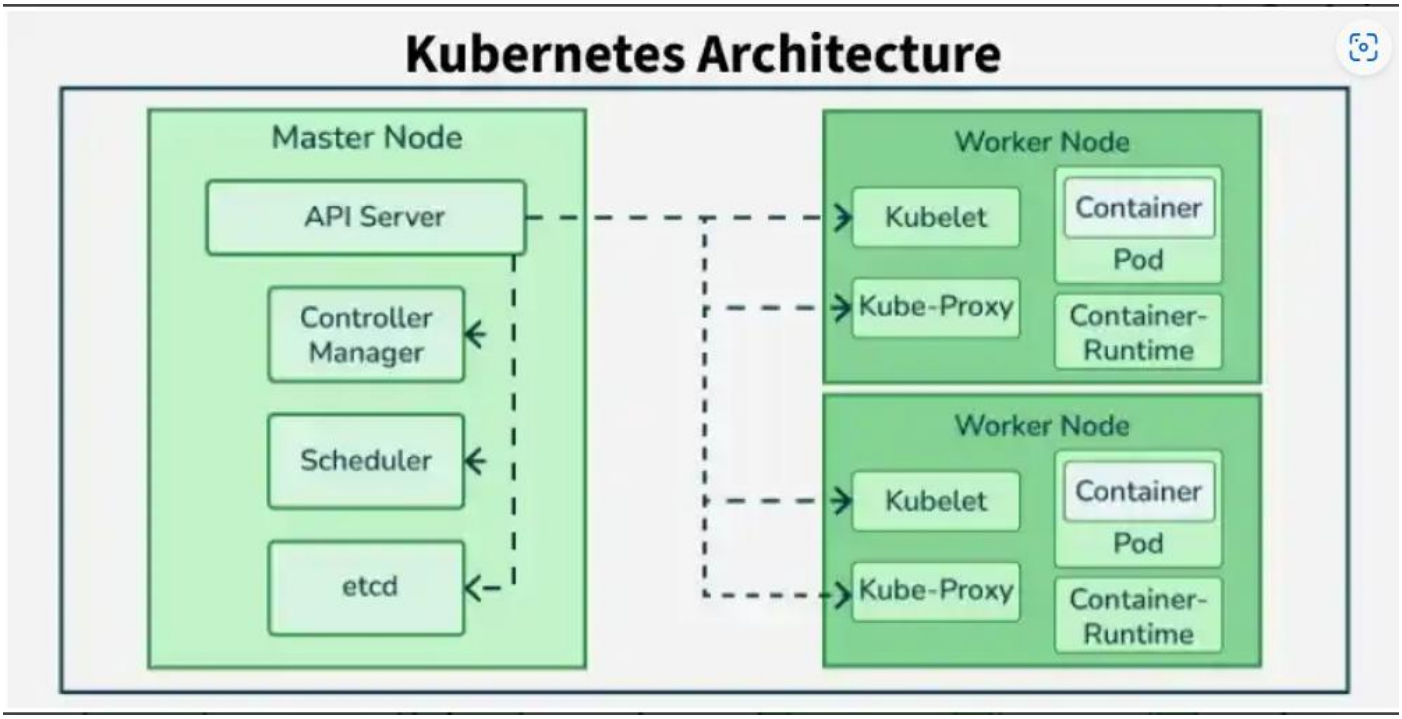
The Control Plane makes global decisions about the cluster (like scheduling applications) and detects/responds to cluster events.

- **kube-apiserver:** The front door to Kubernetes. It exposes the Kubernetes API and is the hub that all other components talk to. Whether you use a command-line tool (like kubectl) or a dashboard, you are talking to this component.
- **etcd:** The cluster's memory. It is a highly available, distributed key-value store that holds all the configuration data and the exact state of everything in the cluster.
- **kube-scheduler:** The matchmaker. It watches for newly created containers (Pods) that have no node assigned to them, and selects the best Worker Node for them to run on based on resources, policies, and constraints.
- **kube-controller-manager:** The regulator. It runs controller processes in the background to regulate the state of the cluster. For example, if a container crashes, the Node Controller notices and ensures a new one is spun up to match your desired setup.
- **cloud-controller-manager:** The bridge to the cloud. This links your cluster into your cloud provider's API (like AWS, Google Cloud, or Azure) to manage things like cloud-based load balancers and storage volumes.

2. Worker Nodes (The Muscle)

Worker Nodes are the actual machines (virtual or physical) where your applications live and run.

- **Kubelet:** The captain of the ship on each node. It is an agent that runs on every worker node, ensuring that the containers are running exactly how the Control Plane requested. If a container goes down, the Kubelet tries to restart it.
- **Kube-proxy:** The traffic cop. It handles the network communication inside and outside of your cluster, making sure network traffic finds its way to the correct container.
- **Container Runtime:** The engine. This is the software responsible for actually running the containers. Kubernetes supports several runtimes, with **containerd** and **CRI-O** being the most common.



Core Objects You Actively Manage

While the components above run the infrastructure, these are the actual building blocks you deploy into Kubernetes:

- **Pods:** The smallest deployable units in Kubernetes. A Pod represents a single instance of a running process in your cluster and contains one or more tightly coupled containers.
- **Services:** Since Pods can die and be replaced frequently, their IP addresses change. A Service acts as a permanent pointer (a stable IP or DNS name) that routes traffic to your Pods, serving as an internal load balancer.
- **Deployments:** This is where you declare the *desired state* of your application (e.g., "I want 3 copies of my web app running at all times"). The deployment controller takes care of updating or scaling your containers automatically.
- **ReplicaSet** is a core Kubernetes object whose sole job is to maintain a stable set of identical Pods running at any given time.

Think of it as a **guaranteed backup and scaling manager**. If you tell Kubernetes, *"I always want exactly 3 identical copies of my API running,"* the ReplicaSet is the component that rolls up its sleeves to make sure that happens.

How a ReplicaSet Works

A ReplicaSet monitors your cluster constantly and uses three key elements to do its job:

1. **Selectors:** It uses labels to identify which Pods it is responsible for managing.
2. **Replicas:** The defined number of Pods that *should* be running (e.g., replicas: 3).
3. **Pod Template:** The blueprint it uses to stamp out new Pods if it needs to create more.

If a worker node crashes and takes two of your Pods down with it, the ReplicaSet instantly notices that the actual count (1) doesn't match the desired count (3). It immediately uses the Pod template to spin up 2 brand-new Pods on a healthy node to bring the count back to 3.

Conversely, if you manually create an extra identical Pod, the ReplicaSet will see that you now have 4 Pods instead of 3, and it will terminate one to maintain the balance.

ReplicaSet vs. Deployment (The Gold Standard)

While ReplicaSets are incredibly useful, you will rarely create them directly. Instead, you will almost always use a **Deployment**.

Here is the hierarchy:

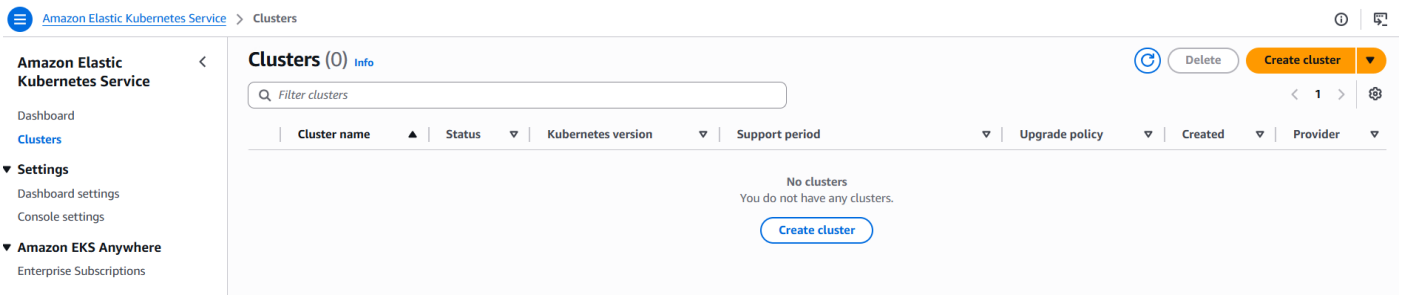
- **Deployment:** Manages the rollout of your application. When you update your application's code or image version, the Deployment automatically creates a *new* ReplicaSet with the new code, scales it up, and scales the *old* ReplicaSet down. This ensures zero-downtime updates.
- **ReplicaSet:** Manages the lifecycle and exact number of the individual Pods.
- **Pod:** The actual container running your code.

Analogy: If a **Pod** is a single worker, a **ReplicaSet** is the shift supervisor making sure the exact right number of workers are on the floor. A **Deployment** is the regional manager, deciding when it's time to train the workers on a brand-new menu or system.

Hands on: Creating a Kubernetes architecture via Ec2 and implementing its components:

Step no:1 To create a K8s architecture we need a need a cluster. Here in AWS we use **EKS cluster.**

Search **EKS → create cluster.**

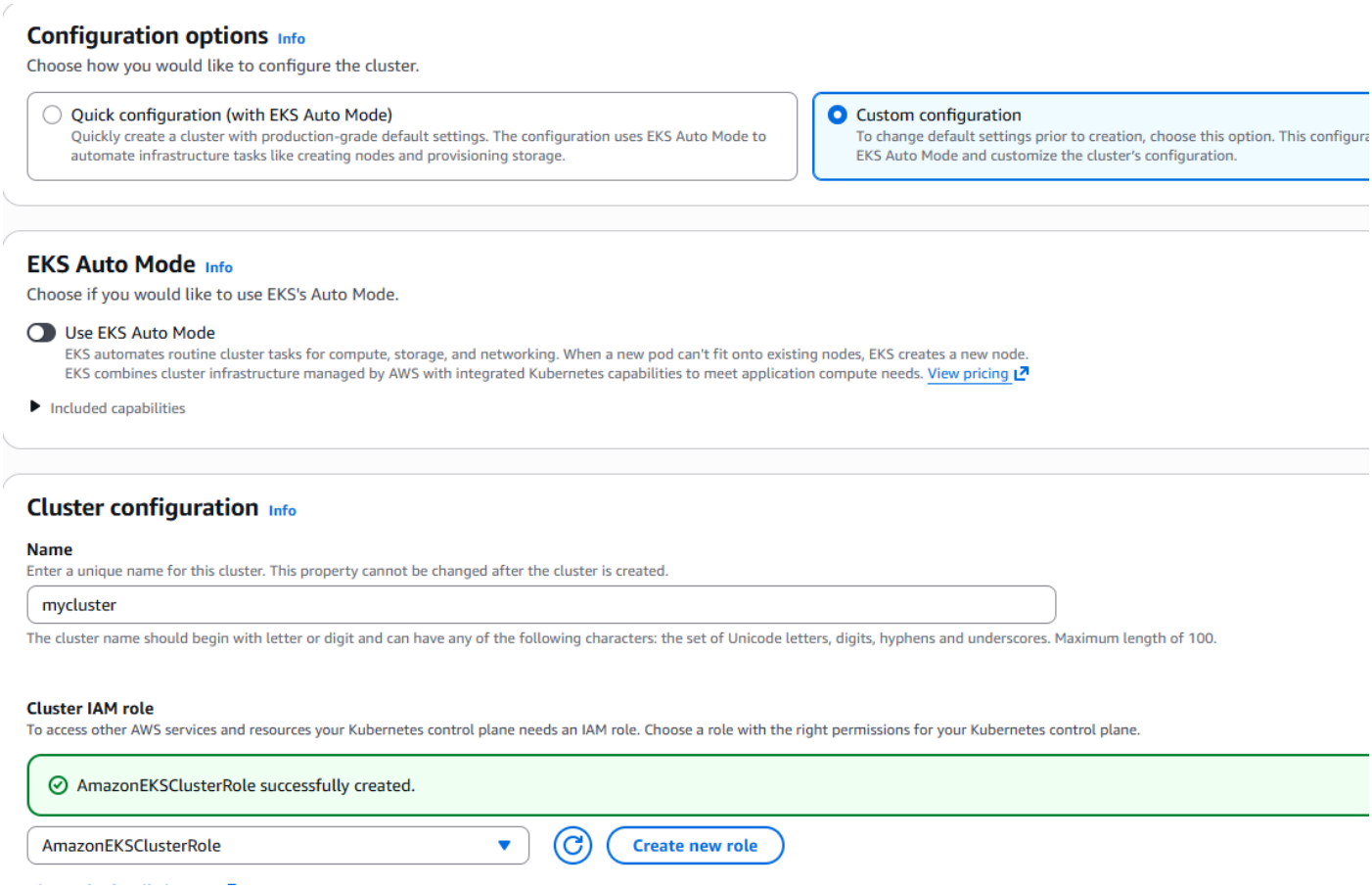


Configuration: **custom configuration**

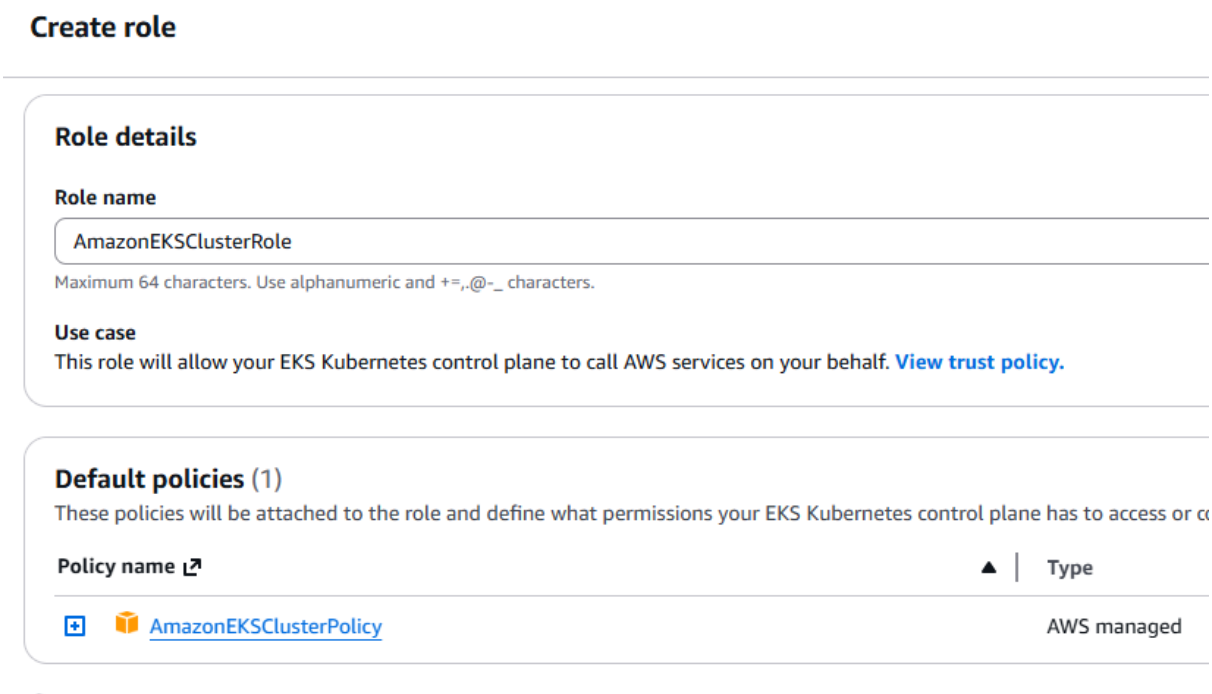
Name: **mycluster**

It will request for an **IAM role** in order to communication with the other AWS resources.

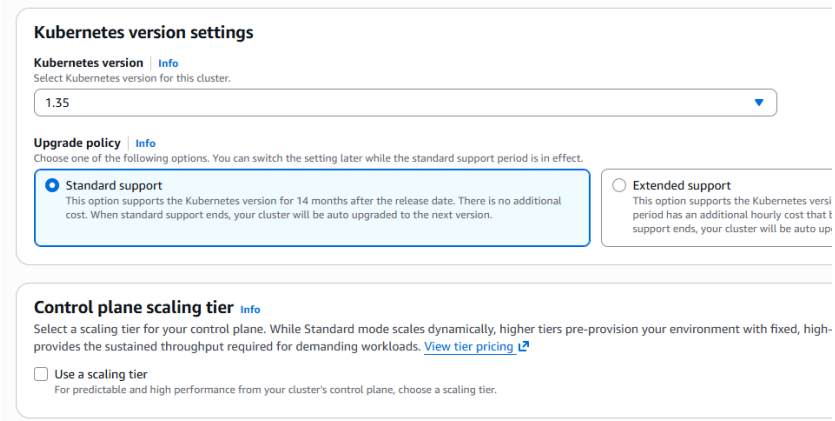
Create role and it will provide the policy needed for the role. Create and choose it.



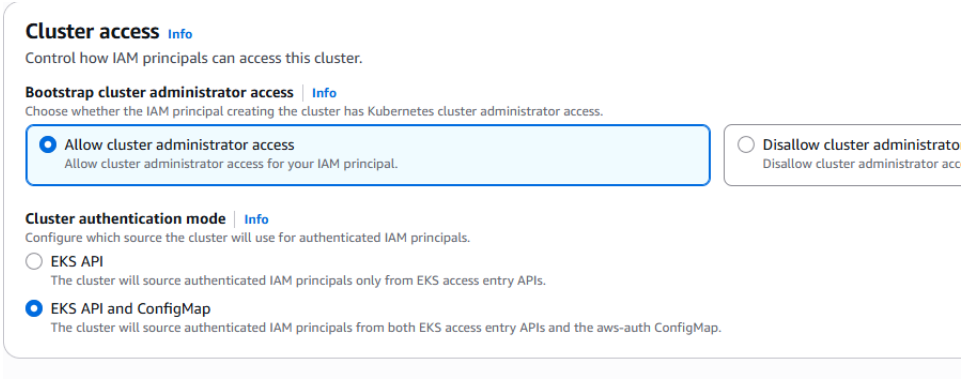
Create role and it will provide the policy needed for the role. Create and choose it. → Next



Keep the **version settings default** and move next.



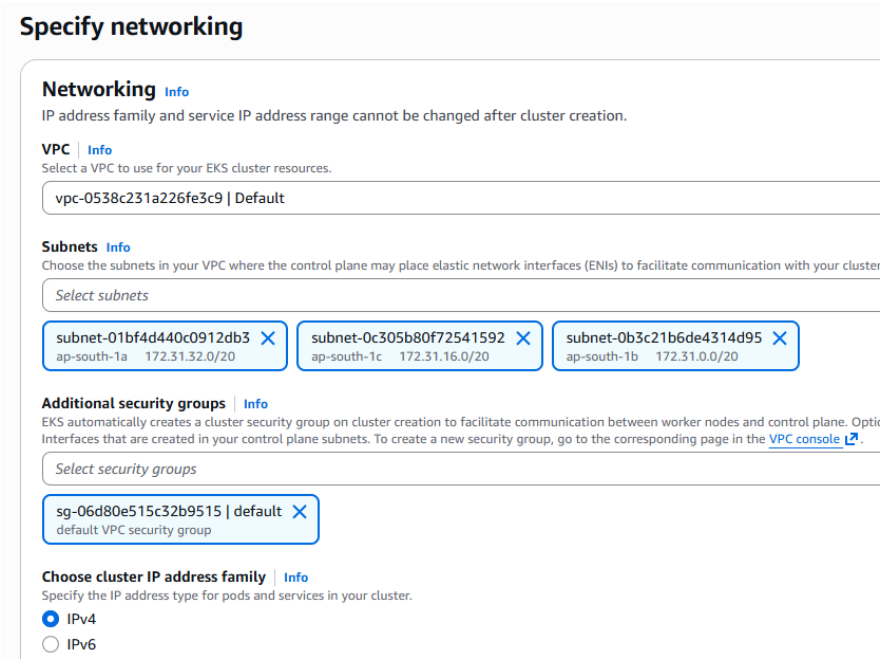
In cluster access choose: **EKS API and ConfigMap**

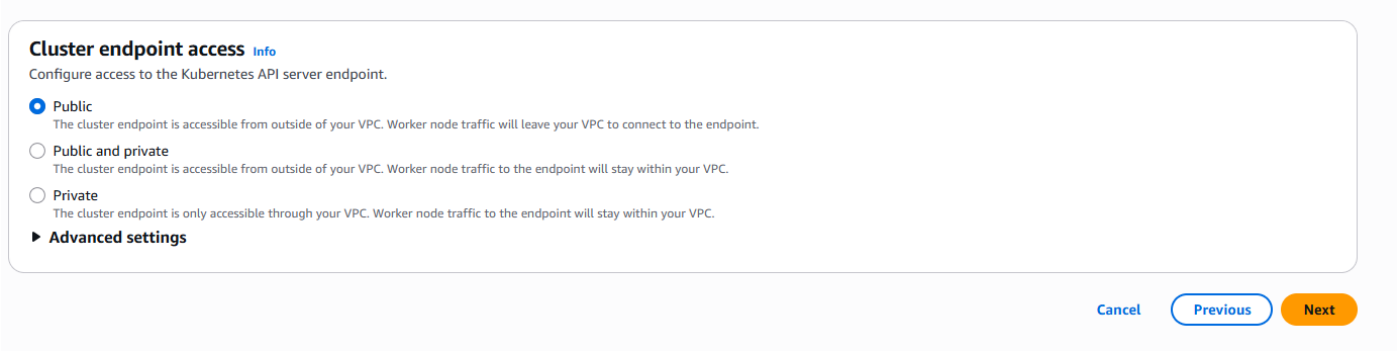


In Network settings,

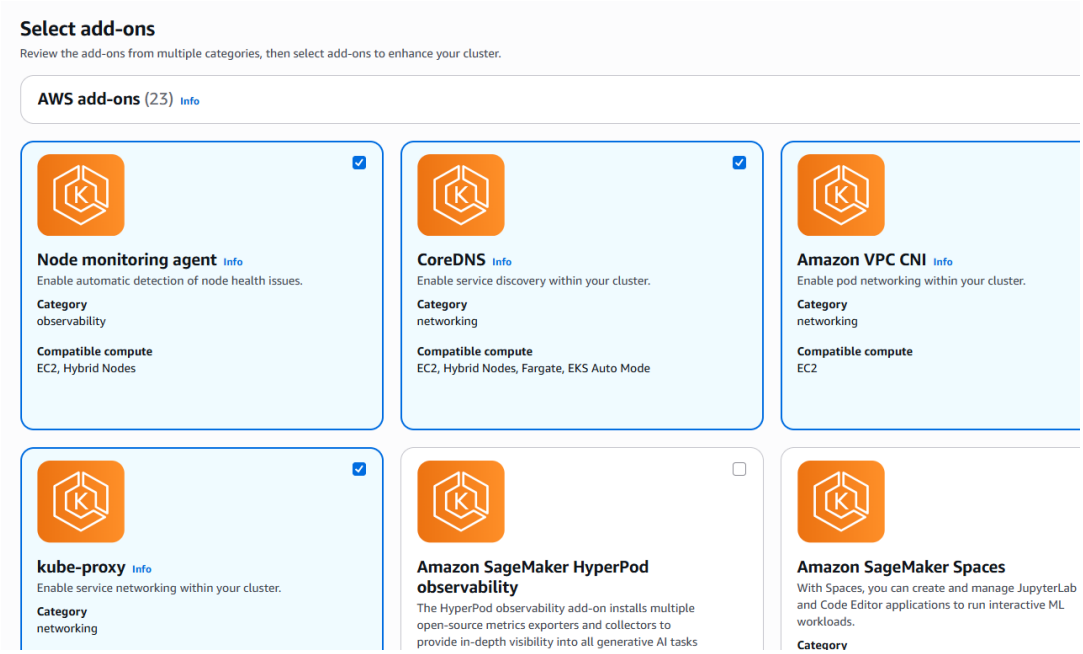
Choose **default VPC** , all the subnets under in and **default SG** (ensure all TCP is enabled).

The **endpoint URL must be public**, so that we can see it from outside the console access as well.

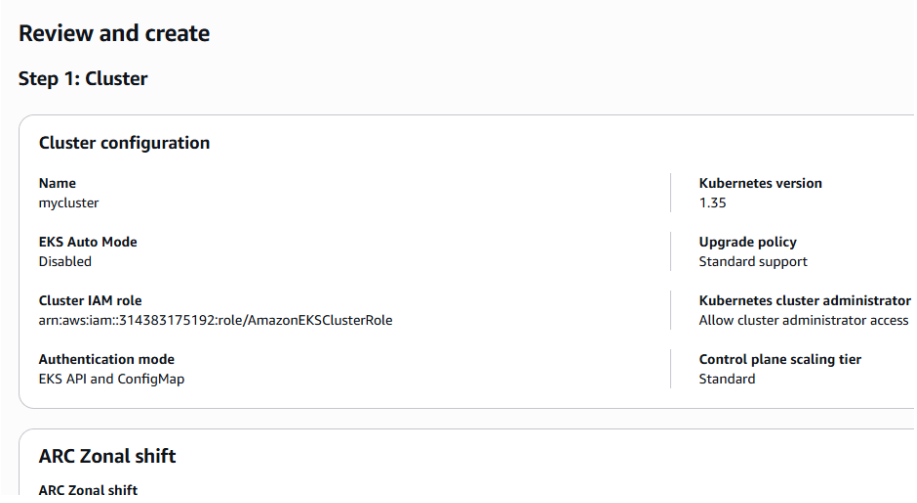




Monitoring: **default ,** Add-ons (plugins of K8s): **default** → Next



Review and click Create.



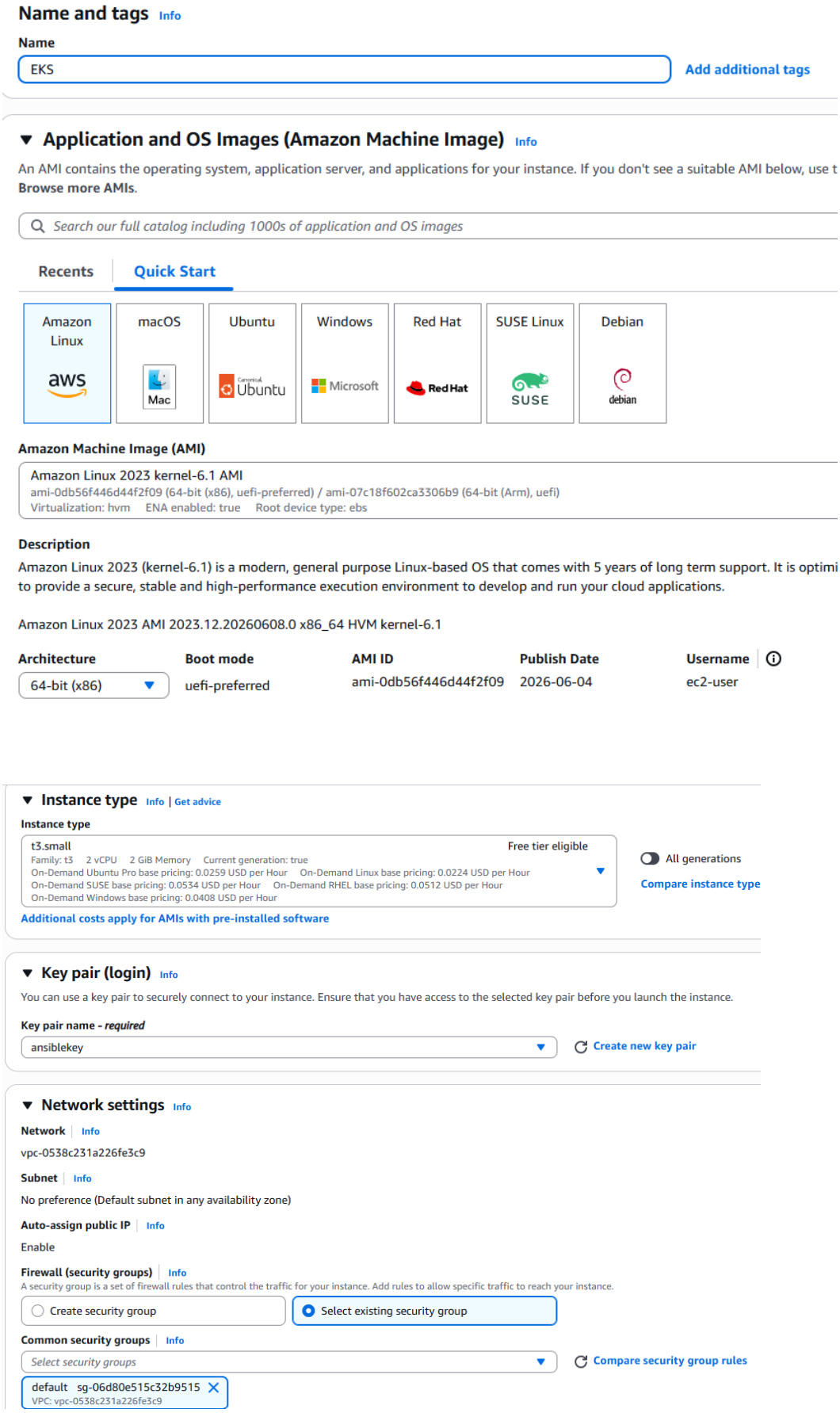
It will take 7 to 8 minutes to create the cluster.

Step no:2 In order to access the cluster we need an instance. This is for communication.

This instance is not the controller. The AWS will take care of the controlling by itself.

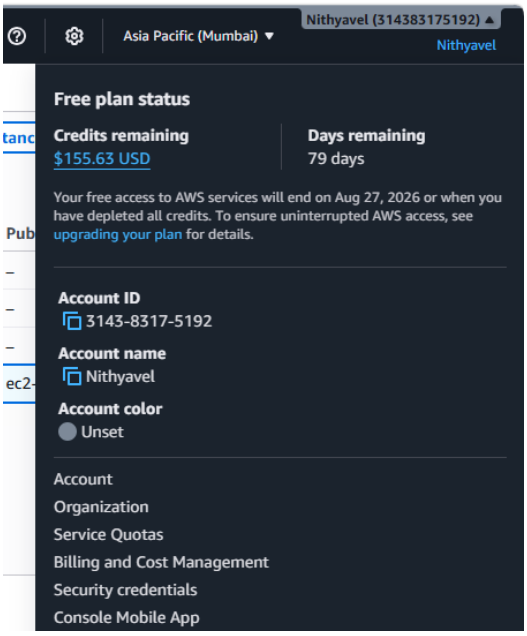
Create an **Amazon Linux instance** with a **key default VPC** and **default SG** (All TCP enabled).

Launch instance.

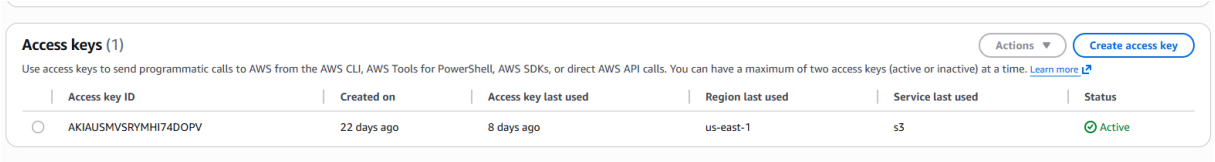


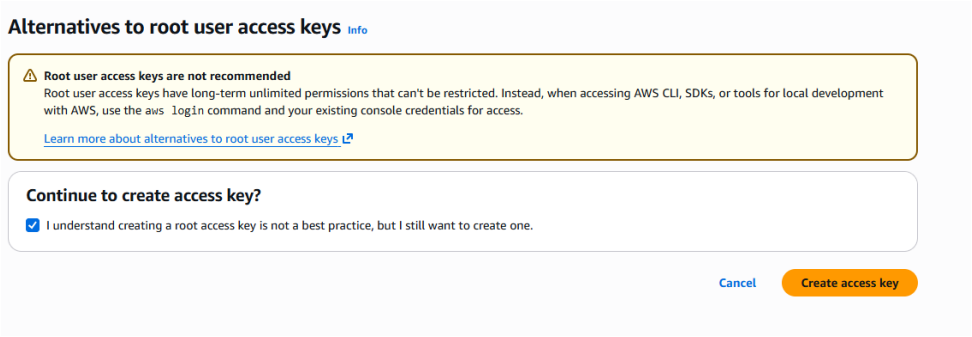
Step no:3 In order to establish the communication between the EKS cluster and the EKS ec2 instance we need **access key and secrete access keys.**

Open **security credentials**

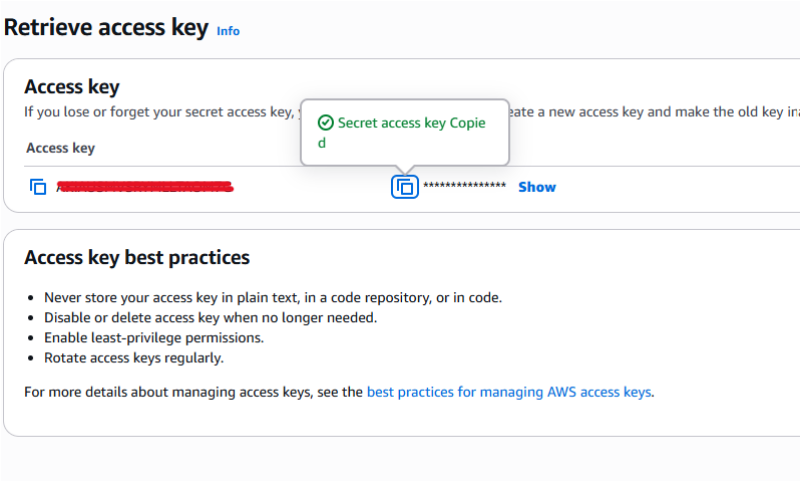


Create access keys





Access key will be created



Now connect our EKS ec2 instance and enter the command to configure the key: **aws configure**

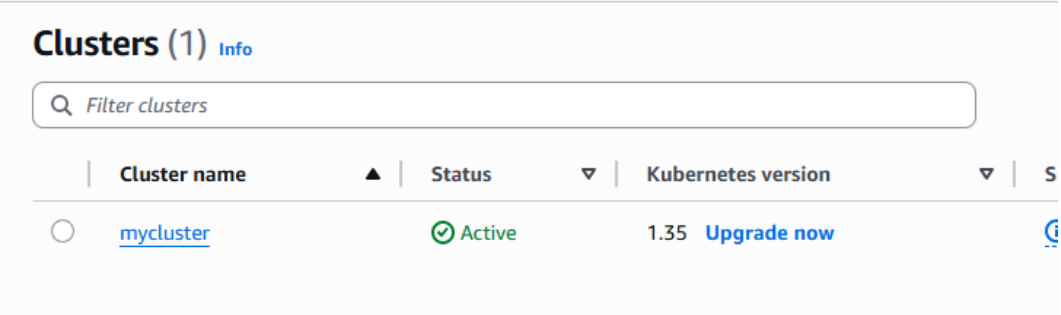
Enter the access key and the secrete access key.

Keep the regions default by just pressing enter.

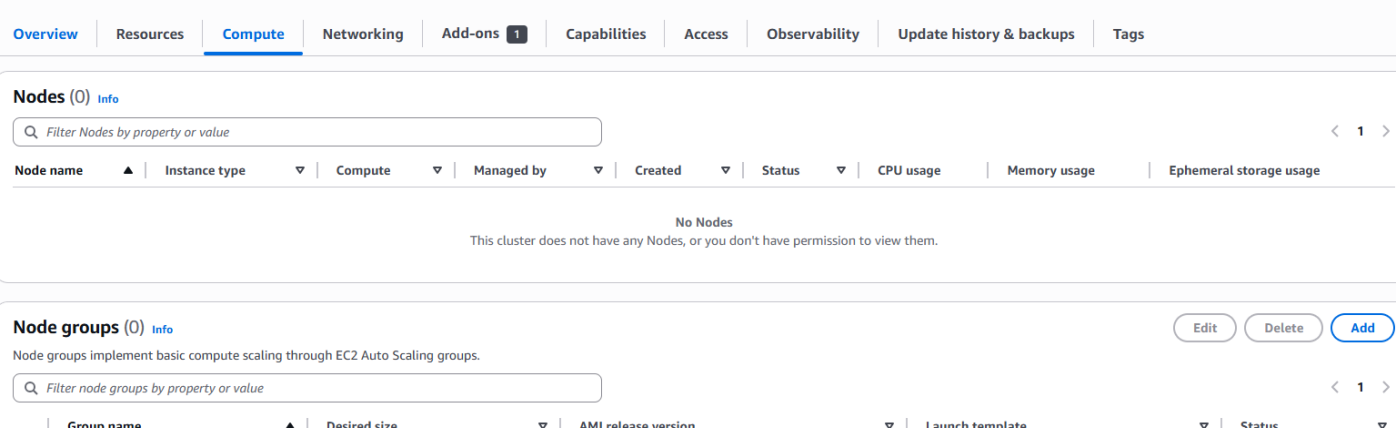
```
       _/ _/
     _/m/'
[ec2-user@ip-172-31-8-71 ~]$ aws configure
AWS Access Key ID [None]: AKIAUSMVSRYMLLTAOM7G
AWS Secret Access Key [None]: gZva9KGrZOVZtBZFOz+D1DrM8
Default region name [None]: Default output format [None]
[ec2-user@ip-172-31-8-71 ~]$
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:4 Now open our cluster we need to have worker nodes inside it in order to perform any tasks.

Our **cluster** is **active** now → click it → Go to **compute**



Under **node groups: click Add**



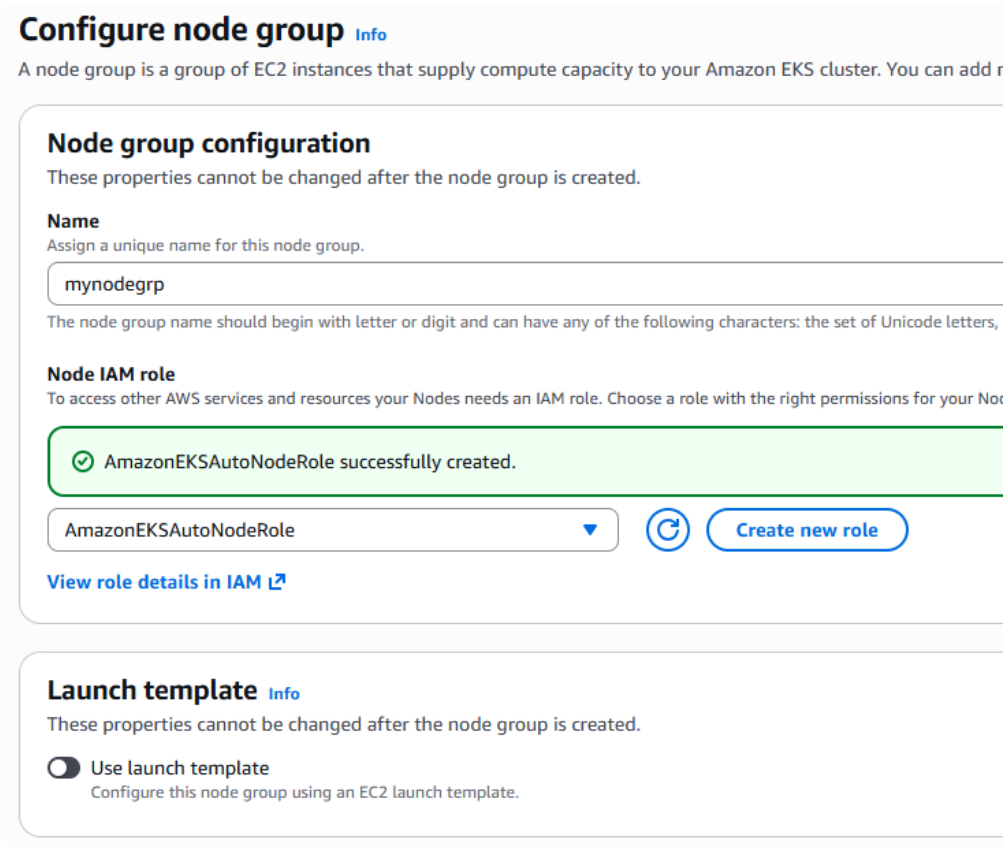
Configure the nodes here. Nodes are also ec2 instances / server.

So it will ask for the instance configurations for the worker nodes.

Name of the node group: **mynodegp**

Here also we need to create an **IAM role → create and select**.

Next

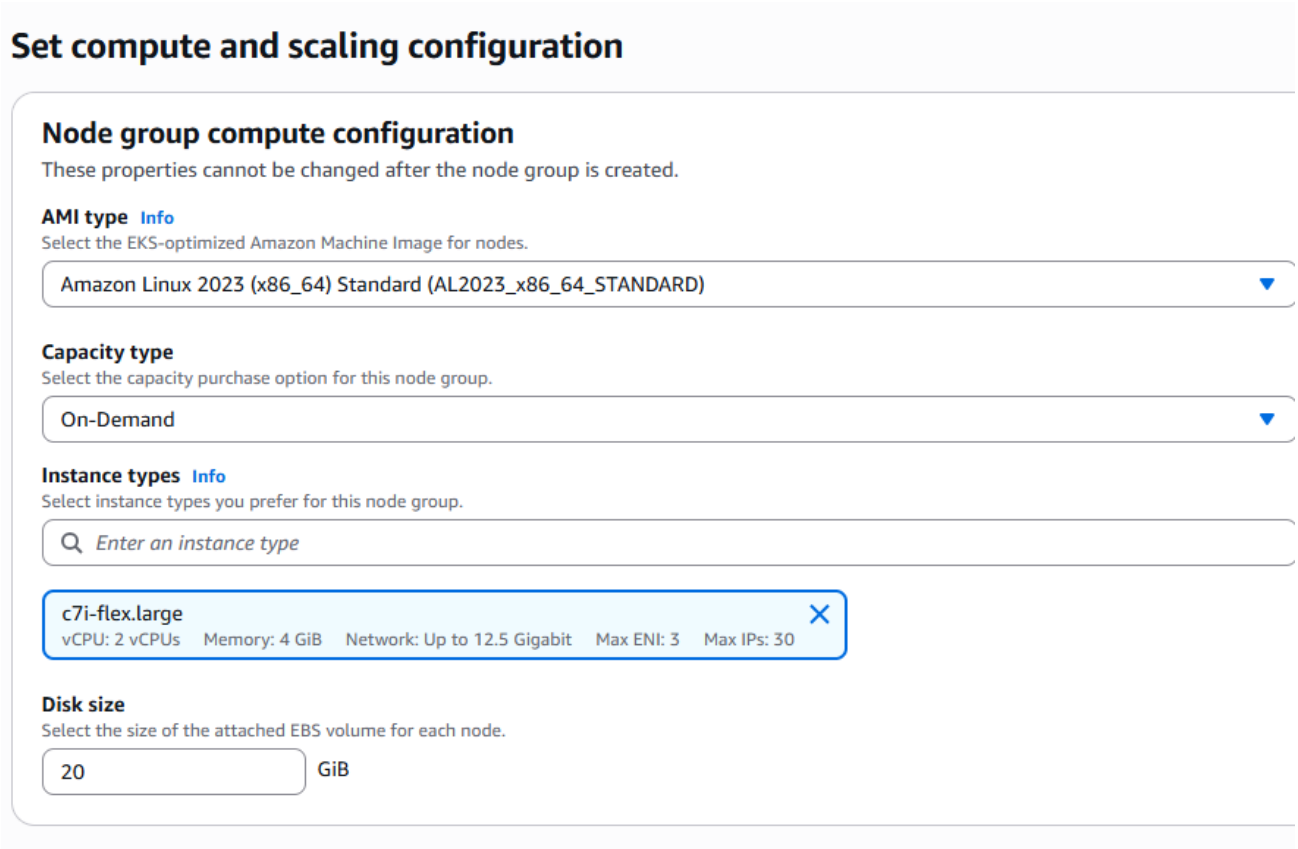


Type of AMI: **Amazon linux**

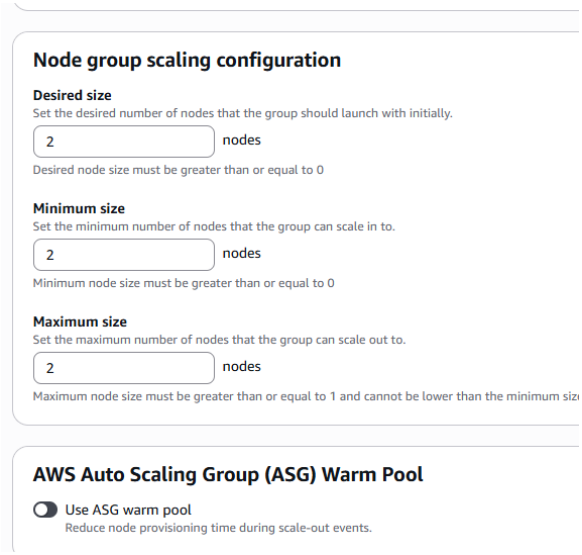
Capacity type: **On-demand**

Instance type: **Check the free tier available** (c7i-flex.large)

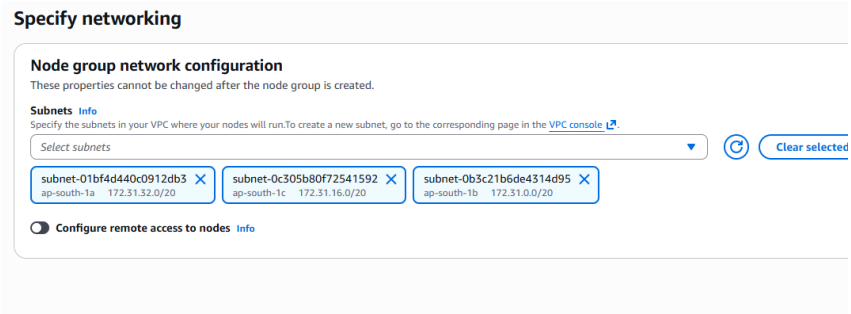
(If we provide other than free tier eligible our worker nodes will not be created)



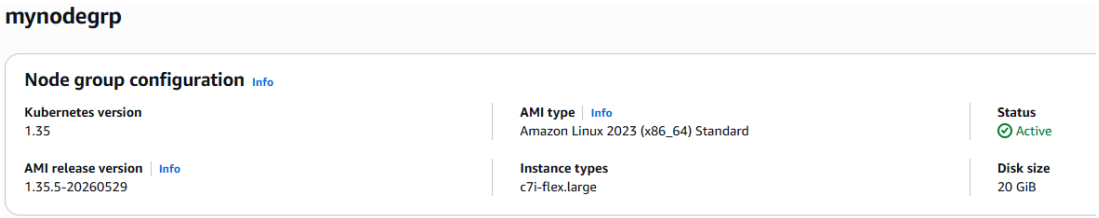
For autoscaling → keep the **default** (maximum size can be increased, can not be decreased)



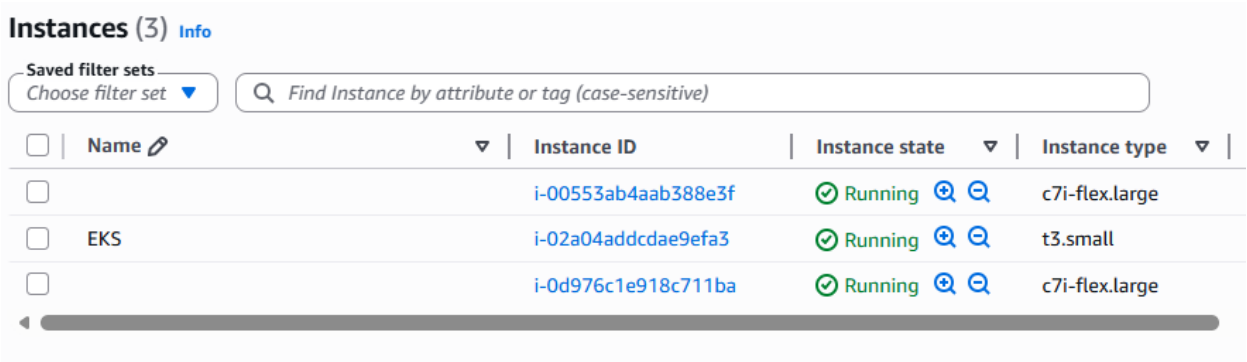
Networking also the default subnets.



Review and create.



Once applied it will take few minutes to create the worker nodes. Once the creation is successful, we can see the two **worker nodes created on demand on the EC2 console page** with the configuration that we have provided. Wait until they are **ready.**



Step no:5 In our EKS instance (Communication instance) we need to make the connection with the two worker nodes we have created. The access keys and secrete access keys are for authentication purpose for communication.

First **install the Kubernetes** in the communication node using the following command:

(git hub repo has this commands)

```
Last login: Wed Jun 10 13:32:27 2026 from 13.233.177.
[ec2-user@ip-172-31-8-71 ~]$ sudo su
[root@ip-172-31-8-71 ec2-user]# curl -LO "https://dl.

chmod +x kubectl
sudo mv kubectl /usr/local/bin/

kubectl version
```

Check the version**: kubectl version**

To establish the connection: **aws eks --region ap-south-1 update-kubeconfig --name** mycluster
(name and location of our cluster)

The nodes will be added

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl version
Client Version: v1.36.1
Kustomize Version: v5.8.1
The connection to the server localhost:8080 was refused - did you specify the right host or port?
[ec2-user@ip-172-31-8-71 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Added new context arn:aws:eks:ap-south-1:314383175192:cluster/mycluster to /home/ec2-user/.kube/config
[ec2-user@ip-172-31-8-71 ~]$
```

We can see the worker nodes from here: **kubectl get nodes**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl get nodes
NAME                                          STATUS   ROLES    AGE
ip-172-31-0-45.ap-south-1.compute.internal    Ready    <none>   82m
ip-172-31-42-214.ap-south-1.compute.internal  Ready    <none>   82m
[ec2-user@ip-172-31-8-71 ~]$
```

In worker nodes we must have pods to run any applications.

Create a node we must have a .yaml file for the node with its configurations and specifications.

Create a file: **vi pod.yaml**

Paste the pod contents. The pod has the containers to run the applications / servers.

```
[ec2-user@ip-172-31-8-71 ~]$ vi pod.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app-pod
  labels:
    app: my-app
spec:
  containers:              # must be 'containers'
    - name: my-container
      image: httpd:latest
      ports:
        - containerPort: 80
```

Now create a pod from the pod.yaml file using: **kubectl apply -f pod.yaml**

We can see the pod using: **kubectl get pod**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f pod.yaml
pod/my-app-pod created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod
NAME         READY   STATUS    RESTARTS   AGE
my-app-pod   1/1     Running   0          16s
[ec2-user@ip-172-31-8-71 ~]$
```

In order to expose the pod to outside world (to see the output) to run as an application we need a service.

Create a service we must have a .yaml file for the service with its configurations and specifications.

Create a file: **services.yaml**

We are using node port service. The node port range is 30000-32767 inside the TCP range.

```
[ec2-user@ip-172-31-8-71 ~]$ vi services.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat services.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: NodePort
  ports:
    - protocol: TCP
      port: 80        # Service port
      targetPort: 80  # Pod container port
      nodePort: 30007 # NodePort (must be between 30000-32767)
```

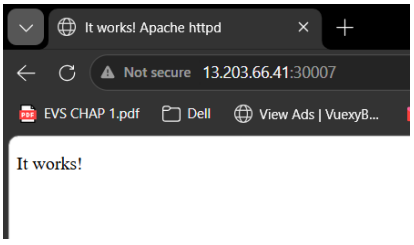
Now create a service from the services.yaml file using: kubectl apply -f services.yaml

We can see the pod using: **kubectl get service**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f services.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get service
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes       ClusterIP   10.100.0.1      <none>        443/TCP        149m
my-app-service   NodePort    10.100.235.66   <none>        80:30007/TCP   10s
```

Now go to the worker nodes and check Default sg all TCP enabled.Http and httpd also because we are running this httpd inside the pod.

Copy the public Ip of any one of the worker nodes:30007 (node port provided in the services.yaml) and paste it in a private browser. We can see the httpd output page



We can see the respective instance in which the pod is running using: **kubectl delete pod my-app-pod**

```
my-app-service   NodePort    10.100.235.66   <none>        80:30007/TCP   10s
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME         READY   STATUS    RESTARTS   AGE     IP              NODE
my-app-pod   1/1     Running   0          6m45s   172.31.39.112   ip-172-31-42-214.ap-south-1.comput
[ec2-user@ip-172-31-8-71 ~]$
```

The pods are ephemeral in nature. If we delete it, it will be dead. It won't create a new pod with this same configuration automatically to overcome the crash by self-healing.

Delete the pod: **kubectl delete pod pod-name**

Delete the service associated with the pod: **kubectl delete svc service-name**

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete pod my-app-pod
pod "my-app-pod" deleted from default namespace
[ec2-user@ip-172-31-8-71 ~]$ kubectl get svc
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)
kubernetes       ClusterIP   10.100.0.1      <none>        443/TCP
my-app-service   NodePort    10.100.235.66   <none>        80:30007/TCP
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete svc my-app-service
service "my-app-service" deleted from default namespace
```

Step no:6 In order to overcome the ephemeral nature of pod, we have deployments which has replica set inside it.If the deployment crashes, it will auto heal and try to create a new one based on the number of replica sets in the .yaml File

Create a deployment file: **vi deployment.yaml** (it has labels: object and selectors (filter to match the lables))

Similarly, the deployment runs the containers.

```
ec2-user@ip-172-31-8-71 ~]$ vi deployment.yaml
ec2-user@ip-172-31-8-71 ~]$ cat deployment.yaml
piVersion: apps/v1
ind: Deployment
etadata:
  name: my-app-deployment
pec:
  replicas: 5
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Apply the deployment.yaml file to create a deployment: **kubectl apply -f deployment.yaml**

Once the deployment has been created. Check the pod info → we can see 5 pods are running (replicas:5 in the deployment.yaml)

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f deployment.yaml
deployment.apps/my-app-deployment created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME                                 READY   STATUS    RESTARTS   AGE   IP
my-app-deployment-58f59c99c-8w5jb    1/1     Running   0          21s   172.31.9.209
my-app-deployment-58f59c99c-dh4rh    1/1     Running   0          21s   172.31.42.95
my-app-deployment-58f59c99c-nvxmj    1/1     Running   0          21s   172.31.46.25
my-app-deployment-58f59c99c-nzs85    1/1     Running   0          21s   172.31.13.110
my-app-deployment-58f59c99c-tp6lb    1/1     Running   0          21s   172.31.32.79
[ec2-user@ip-172-31-8-71 ~]$
```

Delete one of the pods in deployment manually and check the pods. We can see a new one has been autoscaled immediately to match the replica numbers. (within 6s)

```
[ec2-user@ip-172-31-8-71 ~]$ kubectl delete pod my-app-deployment-58f59c99c-8w5jb
pod "my-app-deployment-58f59c99c-8w5jb" deleted from default namespace
[ec2-user@ip-172-31-8-71 ~]$ kubectl get pod -o wide
NAME                                 READY   STATUS    RESTARTS   AGE     IP
my-app-deployment-58f59c99c-dh4rh    1/1     Running   0          2m49s   172.31.42.95
my-app-deployment-58f59c99c-nvxmj    1/1     Running   0          2m49s   172.31.46.25
my-app-deployment-58f59c99c-nzs85    1/1     Running   0          2m49s   172.31.13.110
my-app-deployment-58f59c99c-tp6lb    1/1     Running   0          2m49s   172.31.32.79
my-app-deployment-58f59c99c-tz5f5    1/1     Running   0          6s      172.31.8.128
```

In order to expose the pods inside the deployment to outside world (to see the output) to run as an application we need a service. Create a service we must have a .yaml file for the service with its configurations and specifications.

Create a file: **vi service.yaml**

We are using **Load balancer service**. Make sure that we have deleted the existing service running.

```
[ec2-user@ip-172-31-8-71 ~]$ vi service.yaml
[ec2-user@ip-172-31-8-71 ~]$ cat service.yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  selector:
    app: my-app
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 80        # Service port
      targetPort: 80  # Pod container port
```

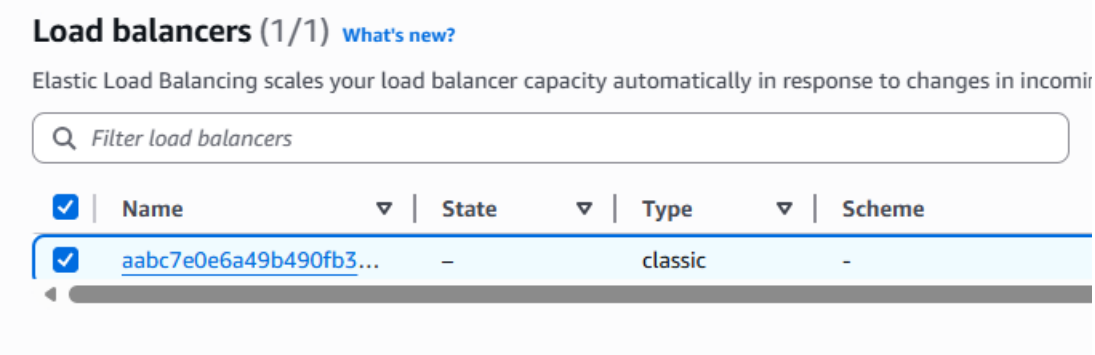
Apply the service**: kubectl apply -f service.yaml**

Check the list of service available. We can see our LoadBalancer service.

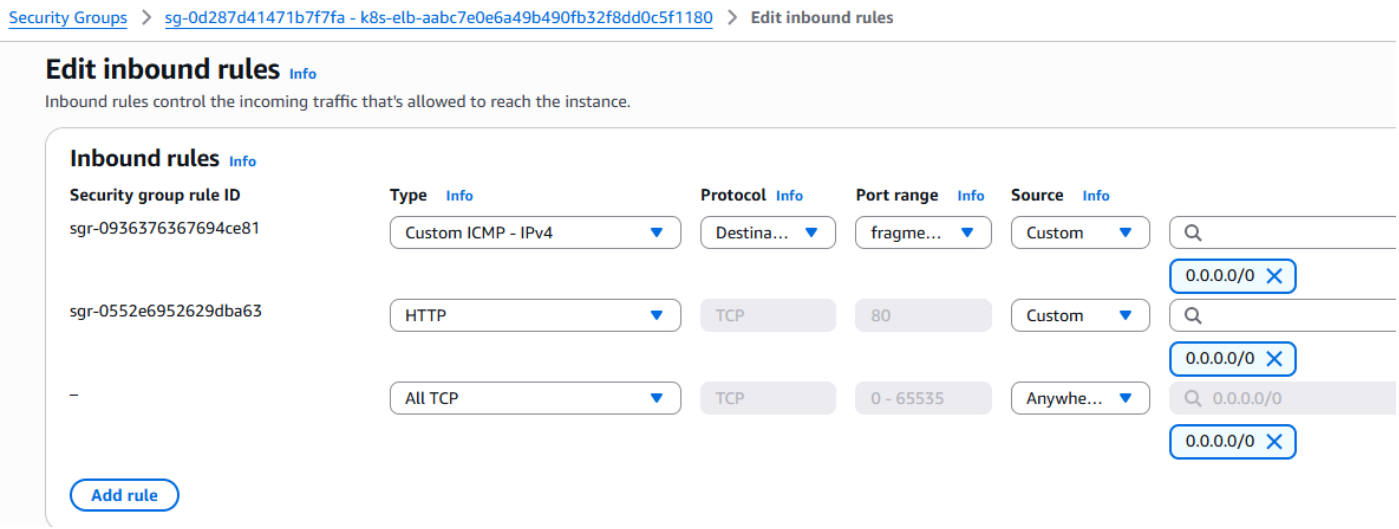
```
[ec2-user@ip-172-31-8-71 ~]$ kubectl apply -f service.yaml
service/my-app-service created
[ec2-user@ip-172-31-8-71 ~]$ kubectl get service
NAME             TYPE           CLUSTER-IP      EXTERNAL-IP
kubernetes       ClusterIP      10.100.0.1      <none>
my-app-service   LoadBalancer   10.100.5.218    aabc7e0e6a49b4
[ec2-user@ip-172-31-8-71 ~]$
```

Step no:7 A load balancer will be created for managing these pods in the deployment.

Go to **Ec2 console → load balancers →** click on **the LB automatically created**.



Security groups → Edit rule → Inbound rules → Add all TCP (check HTTP, ICMP4 present, if not add them also)→save.



Copy the DNS (endpoint url) of the load balancer and paste in **a private browser**. We can see **the nginx page as we are** running nginx inside the pods inside the deployment.